

- ازای نربط ملف js بملف html ؟
- محتاجين الاول نعرف ال browser بيّفهم html ازای ؟

HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title>HTML</title>
  </head>
  <body>
    <h1>Hello World</h1>
  </body>
</html>
```

- الكود ده ازای ال browser بيعرف يترجمه
- اول حاجة بيشفوها ال browser هي

HTML

```
<!DOCTYPE html>
```

- فبيعمل document من النوع html
- بعدها بيشفوف

HTML

```
<html>

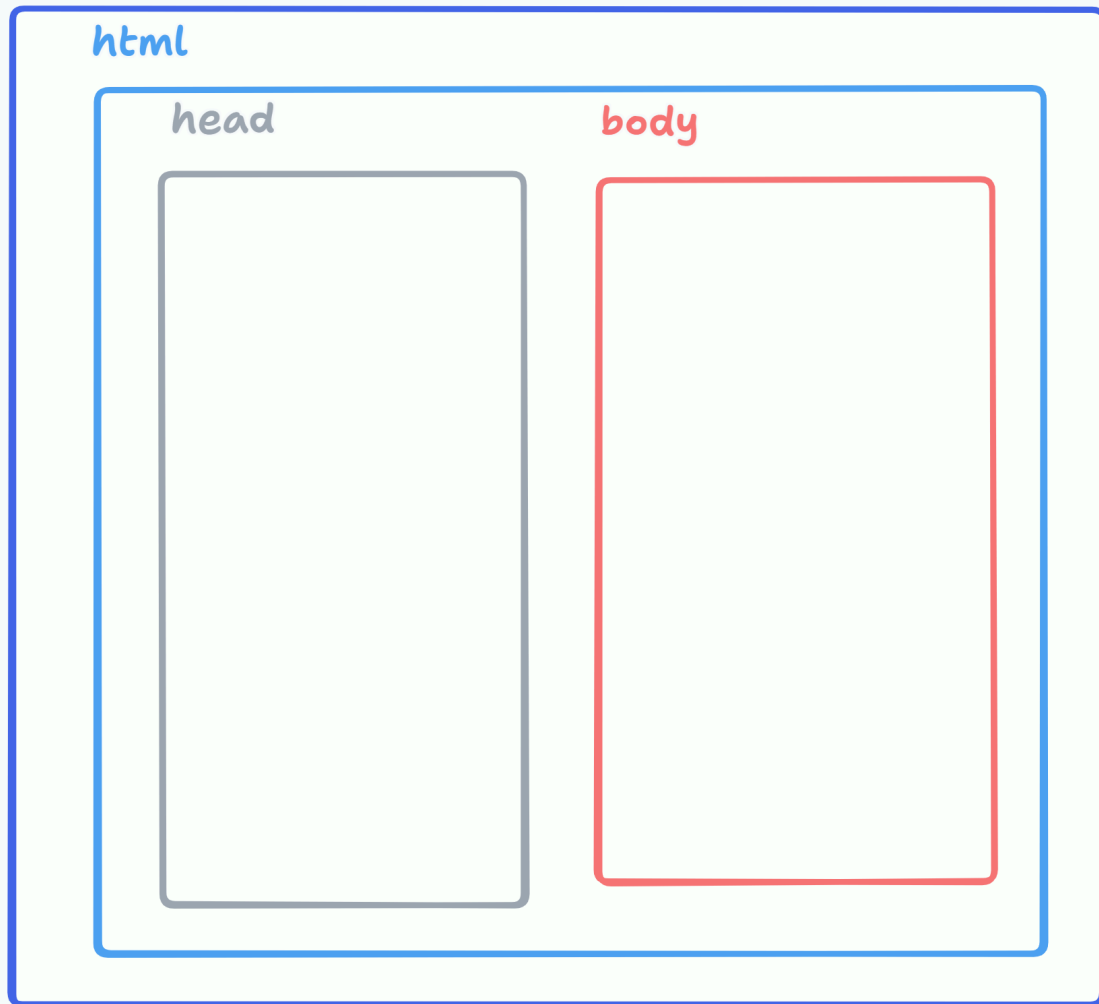
</html>
```

- فبيعمل html جوا ال document

•

- بعدها بيشفوف ال head و محتوياته و يحطهم جوا ال html
- بيخلص ال head و يشوف ال body و محتوياته يحطهم جوا html بردو
- و كدة بيتكون ال document بتاعنا
- بيكون شبه الصورة ده

document



- ده اسمها ال **DOM** اختصار ل document object model و هنتكلم عليها كتير اوي كمان شوية
- اللي محتاجين ناخد بالننا منه دلوقتي ان هي اسمها document object
- يعني ال document في الاصل هي object
- يعني ليها properties و ليها methods
- من خلال ال object اللي اسمه **document** ده بنتحكم بيه في ال document كله
- نقدر نغير في عنصر معين نغير الكلام اللي جواه ال attributes بتاعته ال style بتاعه او نضيف عنصر
- نقدر نتحكم تحكم كامل في ال document عن طريق ال object اللي اسمه **document**

-
-
-

- ال browser فيه مكان برده اسمه ال console
- ال console بيكون موجود في ال developer options يعني حاجة ال user مبيشوفهاش
- ال console هو اداة اليوزر بيعمل بيها debugging للكود بتاعه لاننا نقدر نكتب فيه كود js عادي
- يعني ايه debugging يعني بيتتبع الكود بتاعه عشان يكتشف الازخطاء اللي فيه

- او يجرب فيه مثلا كود معين عشان يشوف هيشغل صح ولا هيبوظ الدنيا قبل ما يحطه في الكود الاصلي
- ال console ده بقى ممكن نتحكم فيه من خلال ال object اللي اسمه `console`
- بردو ال `console` ليه `methods` كتير عارفين منهم طبعا `log()` اللي كنا بنستخدمها طول الفترة اللي فاتت

-
-
-

- طب ازاى اتحكم في ال browser نفسه ؟
- عن طريق object اسمه `window`
- بردو ال object ده بيكون جواه كل حاجات كتير نقدر نتحكم بيها في ال browser مثلا

JAVASCRIPT

```
window.print();
```

- ده ال `print` اللي موجودة في ال browser اللي بتظهر الصفحة اللي بنطبع بيها
- ال `window` جواه كل الاوامر اللي موجودة في js
- معناها ايه الجملة اللي فاتت ده ؟
- معناها ان حتى ال `document` و ال `console` هم اصلا جوا ال `window`
- يعني المفروض نستدعيهم كدة

JAVASCRIPT

```
window.document  
window.console
```

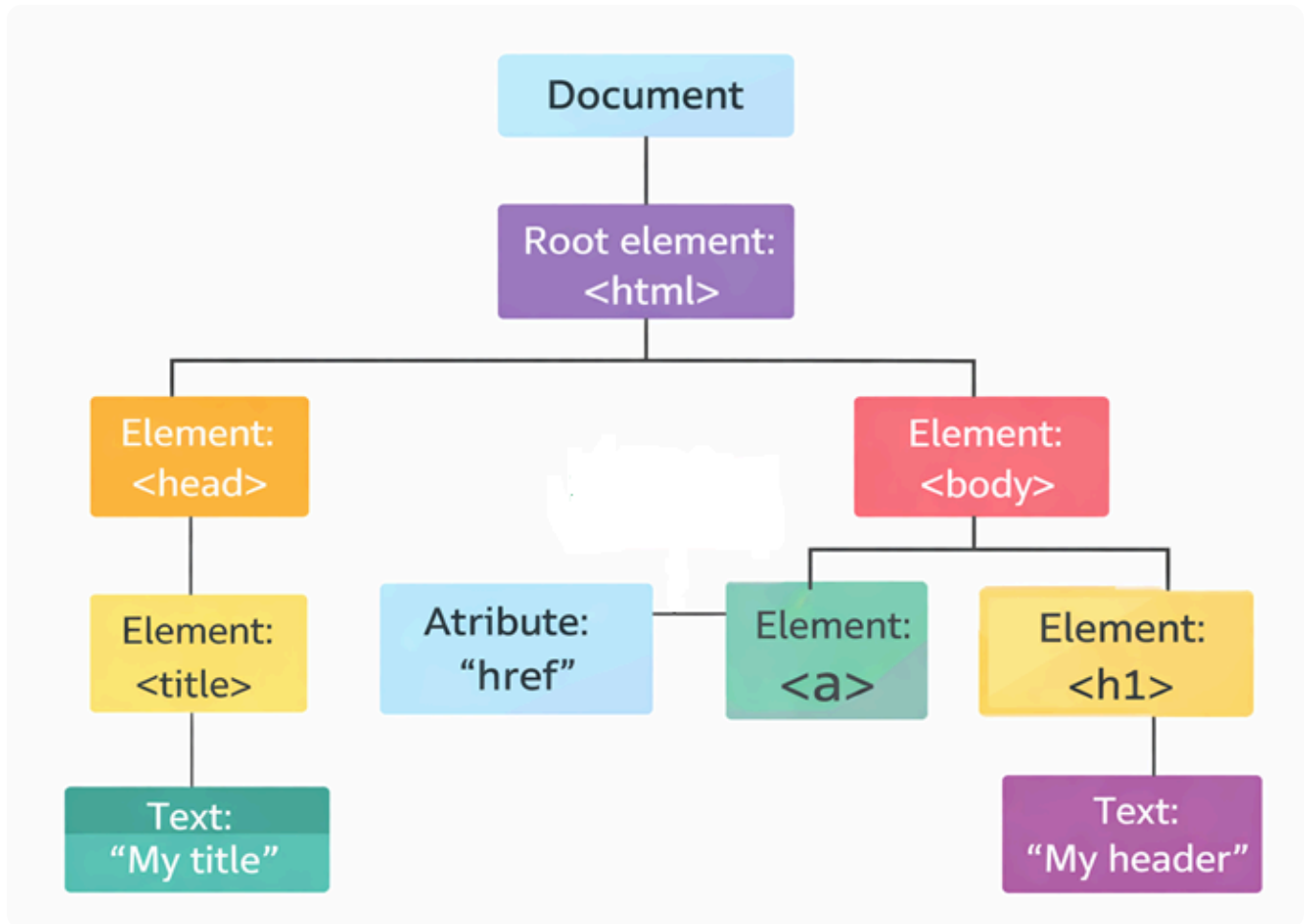
- طب ليه مش بنستدعيهم كدة ؟
- ليه كل الاوامر في js بنستدعيهم من غير `window` ؟
- عشان لما بنستخدم `window` ده معناها اننا جوا ال browser
- طب ما طبيعي اصلا اننا هنكون جوا ال browser
- اي حاجة هنكتبها بتظهر جوا ال browser
- عشان كدة كتبت `window` مكتبتهاش الدنيا شغالة عادي لان هو كدة كدة بيبقى شغل جوا ال browser مش محتاجة تفكره كل شوية انه جوا ال browser بانك تكتبه `window`

٥

DOM

- اتكلمنا من شوية عن ال DOM و قولنا ال browser اول ما بيشوف ملف html بيعمل `document`
- ال `document` ده بيتمثل في js ازاى ؟

- سيتمثل على شكل tree زي الصورة ده كدة



- الشجرة بيبقى ليها فروع بتفضل كل فرع بيبقى جواه فروع تانية لحد ما نوصل في النهاية للورقة
- هي زي كدة بالظبط ال document ليه فرع اساسي اللي هو ال root (الجذع بتاع الشجرة)
- ايه ال root ده بقى بيكون ال html element
- ال html element ده بيطلع منه فرعين
 - واحد بيوصل لل head
 - واحد بيوصل لل body

- ال head بيكون جواه كل ال tags بتاعت ال head كفروع تانية طالعة منه
 - زي title , link , meta
- ال body بيكون جواه ال tags بتاعت ال body كفروع تانية طالعة منه
 - زي h1 , img , p

- ال attributes بتاعت ال tag الواحد بتطلع كفرع منه
- ال text اللي بيتكتب جوا ال tag بردو بيعتبر فرع منه

- كل نقطة بنوصلها اسمها node

- ال tag نفسه اسمه node
- ال attribute اسمه node
- ال text اسمه node كل نقطة بنوصلها اسمها node



Accessing HTML Elements

- احنا قولنا ان `document` عبارة عن object موجود في js
- بنستخدم بال object ده عشان نتحكم في كل ال html elements نعدل فيهم باي شكل
- طب عشان نعدل في element لازم نمسكه الاول
- نمسكه يعني هن access عليه
- كل ال html elements لما ب access عليها بنتعامل معاها ك object بردو يعني ليها properties و methods
- عندنا اكثر من طريقة عشان ن access على html element

-
-

- اول واحدة اننا نمسكه بال id بتاعه

```
<!DOCTYPE html>
<html>
  <head>
    <title>HTML</title>
  </head>
  <body>
    <h1 id="heading">Hello World</h1>
    <script src="main.js"></script>
  </body>
</html>
```

HTML

```
document.getElementById("heading");
```

JAVASCRIPT

- بيكون عندنا html element معين ليه id
- في js بنمسك ال document object و نستدعي واحدة من ال methods اللي جواه
- و اللي هي `getElementById()` و نديها ال id بتاع ال h1 كباراميتير
- الفانكشن ده هترجعنا ال h1 كاربجيكت جواه properties و methods
- كدة احنا مسكنا ال Object بس حرفيا معملناش فيه اي حاجة

- في property مهمة جوا اي html element اسمها `innerHTML`
- ده بتعمل ايه ؟ بيبقى جواها كل ال elements اللي جوا ال element ده سواء text او html elements

```
document.getElementById("heading").innerHTML = "Welcome";
```

JAVASCRIPT

- كدة احنا مسكنا ال Object و مسكنا كل اللي جواه و كمان غيرنا فيه
- المفروض دلوقتي الناتج اللي هيطهر مش Hello World لا هيطهر Welcome عشان احنا غيرنا كل اللي جوا ال element

طب سؤال

لو عندنا ال html كدة

```
<!DOCTYPE html>
<html>
  <head>
    <title>HTML</title>
  </head>
  <body>
    <h1 id="heading">Hello World</h1>
    <h1 id="heading">Hello World</h1>
    <h1 id="heading">Hello World</h1>
    <script src="main.js"></script>
  </body>
</html>
```

HTML

و عملنا نفس الحاجة ب js مسكنا ال element بال id بتاعه و غيرنا اللي جواه

```
document.getElementById("heading").innerHTML = "Welcome";
```

JAVASCRIPT

- ايه اللي هيحصل ؟ انه واحد فيهم اللي هيتغير ؟ ولا كلهم هيتغيرو ؟
- الاجابة ان الاول بس اللي هيتغير و الباقي زي ما هم
- طب ليه ما كلهم ليهم نفس ال id
- عشان js بتفترض انك بتستخدم ال id استخدامه الصح
- ال id لازم يكون مميز و مش متكرر بس انت مكرره
- ف js بتمشي عال tags واحد واحد بالترتيب
- اول ما بتلاقي اول element بال id ده بتمسكه و مبتدورش ثاني خلاص هتدور ليه ما الطبيعي ان ال id مبيتكررش

-
-

- ثاني طريقة ممكن تمسك ال element هي اننا نمسكه باسم ال tag
- عندنا نفس ال html

HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title>HTML</title>
  </head>
  <body>
    <h1 id="heading">Hello World</h1>
    <h1 id="heading">Hello World</h1>
    <h1 id="heading">Hello World</h1>
    <script src="main.js"></script>
  </body>
</html>
```

بس هنستخدم method مختلفة

JAVASCRIPT

```
document.getElementsByTagName("h1");
```

- ملاحظين هنا ان الفانكشن مش زي اللي فاتت هنا اسمها `getElementsByTagName`
- لان طبيعي ال `tag` ممكن نستخدمه اكر من مرة في ال `html`
- فلما نيجي نجيب ال `element` باسم ال `tag` هيرجعنا كل ال `tags` في شكل `array`
- لو جينا نستعمل `innerHTML` هنا مش هتتفع
- ليه ؟ عشان هي بتشتغل على `html element` واحد بس مش `array`
- لو عايزين نغير `element` واحد كل اللي علينا اننا نكتب ال `index` بتاعه

JAVASCRIPT

```
document.getElementsByTagName("h1")[0].innerHTML = "Welcome";
```

- لو عايزين نغيرهم كلهم هنعمل ايه ؟
- حاجة بسيطة اوي خدناها قبل كدة كتير
- مش اللي هيرجع من الفانكشن ده عبارة عن `array`
- الارراي ده ممكن نخزنه في `variable` عادي
- و اي `element` بنخزنه في `variable` عادي
- طب ما تلف عالارراي ب `loop`

JAVASCRIPT

```
let h1Elements = document.getElementsByTagName("h1");

for(let element of h1Elements) {
  element.innerHTML = "Welcome";
}
```

- كدة كل ال `h1 elements` اتغيرو

-
-

- تالت طريقة نقدر نمسك بيها ال element هي ال `()querySelector`
- ده بتعمل ايه ؟ نقدر نجيب بيها ال element سواء بال id او ال tag بتاعه
- لو id هنحط جمب ال id علامة `#`
- لو tag مش هنحط حاجة اسم ال tag زي ما هو

JAVASCRIPT

```
document.querySelector("#heading"); // by id
document.querySelector("h1"); // by Tag name
```

- ال `querySelector` كدة كدة بترجع element واحد بس
- لو هتجيب element بال id فهي كدة كدة بترجع اول element تلاقيه
- لو هتجيب element بال tag فهي برده هترجع اول element تلاقيه مش هترجع ارراي
- طب ليه بنستخدمها و ايه فايدتها ما الاتنين التانيين شغالين
- ال `getElementsByName()` بتشتغل على tag واحد بس
- بمعنى احنا دلوقتي عندنا ال html ده

HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title>HTML</title>
  </head>
  <body>
    
    <div>
      
    </div>
    <script src="main.js"></script>
  </body>
</html>
```

- عايز اوصل لل img اللي جوا ال div
- استخدمنا ال `getElementsByName("img")` هترجع كل ال img موجودة في ال document
- بس احنا عايزين واحدة معينة فممكن نجيبها كدة

JAVASCRIPT

```
document.querySelector("div img");
```

- كدة مش هيرجعلنا غير ال img اللي جوا ال div

-
-

- رابع طريقة و هي مش جديدة اوي
- هي زيه زي ال `querySelector` بس بترجع ارراي فيها كل ال elements

```
document.querySelectorAll("img");
```

JAVASCRIPT

- ده كدة هيرجعلي كل الصور اللي موجودة في ال document في ارراي
- نرجع ل html اللي في الاول خالص

```
<!DOCTYPE html>
<html>
  <head>
    <title>HTML</title>
  </head>
  <body>
    <h1 id="heading">Hello World</h1>
    <h1 id="heading">Hello World</h1>
    <h1 id="heading">Hello World</h1>
    <script src="main.js"></script>
  </body>
</html>
```

HTML

```
document.querySelectorAll("#heading");
```

JAVASCRIPT

- المفروض الكود ده يطلع ايه ؟
- هيرجع ارراي فيه الثلاثة h1 اللي واخدين ال id ده
- مش بيتعامل على اساس ان ال id المفروض ميكرررش و كدة لا ملوش دعوة
- اي element واخذ ال id ده هيرجعه
- دلوقتي انه فيهم الاحسن ؟ مفيش طريقة احسن من الثاني انت بتستخدم على حسب احتياجك

س

Accessing HTML Attributes

- في طريقتين ن access بيهم عال attributes بتاعت ال html elements
- اول طريقة بتقول ان ال html element زي ماحنا عارفين هو عبارة عن object في js

- ال attributes المشهورة زي `id` , `src` , `value` و في غيرهم كتير بتكون موجودة على شكل property في ال object ده
- مثال :

```
<!DOCTYPE html>
<html>
  <head>
    <title>HTML</title>
  </head>
  <body>
    <input type="text">
    <script src="main.js"></script>
  </body>
</html>
```

```
const input = document.getElementsByTagName('input')[0];
input.value = "Shams";
console.log(input.value);
```

- اول حاجة احنا جيبنا ال element
- و احنا عارفين ان لو عايزين نحط قيمة default في ال input بنحطها ب attribute اللي اسمه `value`
- و بردو احنا لو عايزين ناخذ القيمة اللي اليوزر دخلها خلاص ممكن ناخذها ب `value` بردو
- دلوقتي بقى معنا ال input element هنستخدم ال `.` عشان نجيب ال property بتاعت ال `value`
- دلوقتي يقدر يغير فيها براحته
- ممكن بردو ناخذ القيمة اللي جواها نستخدمها في اي حاجة
-
-

- الطريقة الثانية هنستخدم فانكشن `setAttribute()`
- الفانكشن ده هتاخذ قيمتين string كبار اميترز
- اول قيمة هي عبارة عن ال attribute نفسه
- ثاني قيمة عبارة عن القيمة بتاعت ال attribute ده
- الطريقة ده شغالة مع ال attributes المشهورة و اللي مش مشهورة
- هنجربها على نفس اللي فوق

```
const input = document.getElementsByTagName('input')[0];
input.setAttribute("value", "Shams");
```

- لو عايزين نجيب قيمة ال attribute هنعملها بفانكشن ثانية

```
console.log(input.getAttribute("value"));
```

طب ليه موجودين الاتنين ؟ هم يعتبر بيعملو نفس الحاجة
الاجابة ان لا الاتنين مبيعملوش نفس الحاجة
عايزين نفرق بين حاجتين

1. HTML Attribute
2. DOM Property

ال HTML Attributes ده القيم اللي الصفحة بدات بيها
ال DOM Property ده بتمثل اللي اليوزر شافيه دلوقتي
يعني مثلا

```
<!DOCTYPE html>
<html>
  <head>
    <title>HTML</title>
  </head>
  <body>
    <input type="text" value="Shams">
    <script src="main.js"></script>
  </body>
</html>
```

```
const input = document.getElementsByTagName('input')[0];
input.value = "SS";
console.log(input.value); // SS
console.log(input.getAttribute("value")); // Shams
```

- ال property بتعبر عن القيم اللي اليوزر شافيهها دلوقتي و التغييرات اللي بتحصل ادم اليوزر
- لكن القيمة الابتدائية لل attribute بتفضل زي ما هي عمرها ما بتتغير الا لو انت غيرتها بايدك من html
- او غيرتها ب `()setAttribute` لانها معمولة عشان تغير في الاصل
- ال `()getAttribute` بتجيب الاصل اللي مبيتغيرش او بيتغير بال `()setAttribute`
- ملهاش دعوة باللي بتعمله ال property و تفاعل اليوزر مع الصفحة

- ال event هو عبارة عن حدث
- يعني كود js هيتنفذ لما يحصل حاجة معينة
- هو طريقة لربط ال elements بتاعت html بكود js

```
<element event="js code">...</element>
```

HTML

- اشهر event عندنا اسمه `onclick` بينفذ كود js معين اول ما نضغط عال element ده

```
<button onclick="console.log('hello')">Click</button>
```

HTML

- دلوقتي كل ما نضغط عالزرار هيطبعلنا كلمة hello في ال console
- احنا عارفين لما بنيجي نكتب string في js ممكن نكتبه جوا " " او ' '
- لو لاحظنا جوا ال `console.log()` كتبنا الكلمة جوا ' ' بس دلوقتي احنا مضطرين ليها
- لان لو استخدمنا " " هيطلع ايرور

- طب دلوقتي هو ال event بينفذ حاجات في ال console بس
- لا ما قولنا اننا نقدر نكتب اي كود js جوا يعني ممكن غير اي حاجة في الصفحة

```
<button onclick="document.getElementById('btn').innerHTML = 'Thank You';" id="btn">Click
```

HTML

- بدل ما نقعد نجيب ال element اللي احنا فيه بال id بتاعه في طريقة ابسط
- مش احنا عارفين ان js بتتعامل مع html elements ك object
- و احنا دلوقتي حرفيا يعتبر احنا جوا ال object
- يبقى بما اننا قادرين نتعامل ك object يبقى نقدر نستخدم `this` و احنا عارفين ان `this` بتشاور عال object اللي هي فيه
- يبقى نقدر نحول الكود لكدة

```
<button onclick="this.innerHTML = 'Thank You';" id="btn">Click</button>
```

HTML

- هيشغل زيه زي اللي قبله بالطبط
- طبعا الموضوع ده هيشغل لو كود js هيتنفذ عال element ده بس
- لو هيتنفذ على element تاني اكيد مش هنستخدم `this` لانها بتشاور عال element اللي احنا فيه بس

- بمناسبة `this` احنا قولنا انها بتشاور عال object اللي هي مكتوبة فيه
- طب الكود ده هيكون الناتج بتاعه ايه ؟

```
console.log(this);
```

JAVASCRIPT

- الاجابة `this` هنا هتتشاور على window object
- لان احنا قولنا في الاول ان الكود تلقائي بيعتبر انه جوا window object

نرجع تاني لل event

- المساحة اللي هنكتب فيها كود js الى حد ما قليلة
- الكود اللي هنكتبه لو كبير هيتكتب بشكل مش clean خالص و مش هيتقري بسهولة
- و كمان دايمًا يفضل اننا نفصل ال logic عن ال structure فايه الحل لكدة
- ماحنا ممكن جدا نعمل فانكشن بكل اللي محتاجينه يتعمل و نستدعي بس الفانكشن ده في html

```
<button onclick="sayThankYou()" id="btn">Click</button>
```

HTML

```
function sayThankYou() {
  document.getElementById("btn").innerHTML = "Thank You!";
}
```

JAVASCRIPT

- و ممكن نسيبنا من الكتابة في html خالص و نعمل كل حاجة في js بالطريقة ده

```
const btn = document.getElementById("btn");

btn.onclick = function () {
  document.getElementById("btn").innerHTML = "Thank You!";
}
```

JAVASCRIPT

- او لو الفانكشن معموله كدة هنديه اسمها بس

```
btn.onclick = sayThankYou;
```

JAVASCRIPT

- في events تانية كتير ممكن نستعملها غير `onclick`
- زي مثلا `onmouseover` بتنفذ فانكشن اول ما نقف عال element بالماوس

```
btn.onmouseover = sayThankYou();
```

JAVASCRIPT

- و في event تاني اسمه `onmouseout` لما الماوس ميكونش عال element
- ممكن نستغلها في اننا نرجع العنصر زي ما كان لما نبعد عنه بالماوس

```
btn.onmouseout = () => {
  btn.innerHTML = "Click";
}
```

JAVASCRIPT

Event Listener

- بتعمل نفس اللي بتعمله ال `onevent` عموما بس هي عبارة عن فانكشن بتاخذ callback function

```
element.addEventListener(event, callback);
```

JAVASCRIPT

- نجربها بردو عالزرار اللي عملناه

```
btn.addEventListener('click', function () {
    btn.innerHTML = "Thank You";
});
```

JAVASCRIPT

- تاخذ اول باراميتر اللي هو ال event و ثاني باراميتر هو ال function اللي هتتنفذ لما يحصل ال event ده
- نجربها عال events الثانية اللي قولنا عليها اول حاجة `mouseover`

```
btn.addEventListener('mouseover', function () {
    btn.innerHTML = "Thank You";
});
```

JAVASCRIPT

- نجرب `mouseout`

```
btn.addEventListener('mouseout', function () {
    btn.innerHTML = "Click";
});
```

JAVASCRIPT

- طب ليه في اثنين مع انهم بيعملو نفس الحاجة
- اول سبب ان ال `onevent` متقدرش تنفذ اكر من function في وقت واحد
- لكن `addEventListener` تقدر تعمل كدة

```
btn.onclick = function () {
    console.log("first");
}
btn.onclick = function () {
    console.log("second");
}
btn.onclick = function () {
```

JAVASCRIPT

```
console.log("third");  
}
```

- الناتج من الكود ده هيكون

third

- ليه منفذش باقي الفانكشنز ؟
- ال `btn` هو عبارة عن `object`
- و `onclick` هي عبارة عن `property` جوا ال `object`
- و ال `property` هي زي ال `variable` كدة كل ما نحط جواها قيمة اللي قبلها بتخفي
- فلما بندوس عالزرار اخر فانكشن بتكون متسجلة جواه هي اللي بتتنفذ و الباقي ملهمش وجود

```
btn.addEventListener('click', function () {  
    console.log("first");  
});  
btn.addEventListener('click', function () {  
    console.log("second");  
});  
btn.addEventListener('click', function () {  
    console.log("third");  
});
```

JAVASCRIPT

first
second
third

- ال `addEventListener` نفذت الفانكشنز كلها
- ليه ؟ لان هي بتجمع الفانكشنز بتاعت ال `event` في ارراي و تنفذهم واحدة واحدة
- ثاني سبب
- لو جربنا منحطش فانكشن في `addEventListener` هيطلعنا ايرور علطول
- ليه ؟ عشان احنا بنديها `callback` فهي هتتنفذها في وقت معين و لان اللي اديناها مش فانكشن
- فلما تيجي تنفذ هيطلع ايرور علطول يقول ان اللي جوا ده مش فانكشن عشان ننفذه

•

- طب بالنسبة ل `onevent` مصممة بطريقة مختلفة شوية
- لو خدنا بالنسبة للفرق بين ال `onevent` ك `attribute` و ك `property`
- ال `onevent` ال `attribute` اللي بتتخط في `html` كتبنا فيها كود علطول
- و ممكن عادي نكون عاملين فانكشن معينة و نديها تنفذها
- لكن في ال `onevent` ال `property` بنديها فانكشن

-
- طب ليه ؟
- احنا عرفنا فوق ان ال property مش هي هي ال attribute هم مختلفين
- ال **onevent** ال attribute ال browser بيشفو اللي جواها ككود js و بينفذه علطول اول ما يحصل ال event
- انما ال **onevent** ال property هي property وظيفتها انها يتخزن جواها فانكشن
- فالتبعية ان لو اتخزن حاجة غير الفانكشن هيجصل مشاكل كتير

طب نجرب

```
<button onclick="alert('button')" id="btn">Click</button>
```

HTML

- اللي جوا ده كود js فال browser هينفذه عادي

```
<button onclick="test" id="btn">Click</button>
```

HTML

- اللي جوا مش كود js فلما ال browser يحاول ينفذه مش هيعرف فهيطلع ايرور

- طب بالنسبة لل property

```
btn.onclick = () => {
  alert('button');
}
```

JAVASCRIPT

- هيتنفذ عادي لان اللي جواها فانكشن

- طب كدة

```
btn.onclick = "test";
```

JAVASCRIPT

- ايه اللي هيجصل ؟
- مش هيتنفذ في نفس الوقت مش هيطلع ايرور
- طب ليه ؟
- عشان ميحصلش ايرور ف js بتشفو الاول اللي بنحطه ده فانكشن ولا لا
- لو فانكشن هتخطه عادي و الكود هيشغل تمام
- طب لو مش فانكشن هتخط مكانه **null** فبيبقى ولا كانه موجود
- لما يحصل بقى ال event و ال browser بييجي ينفذ اللي جوا ال **onevent** ده هيجصل ايه ؟
- ولا حاجة هنلاقي جواه **null** و هي برده عبارة عن كود js فهينفذهها و خلاص مش هيطلع ايرور

- لو جربنا نطبع اللي جوا ال property

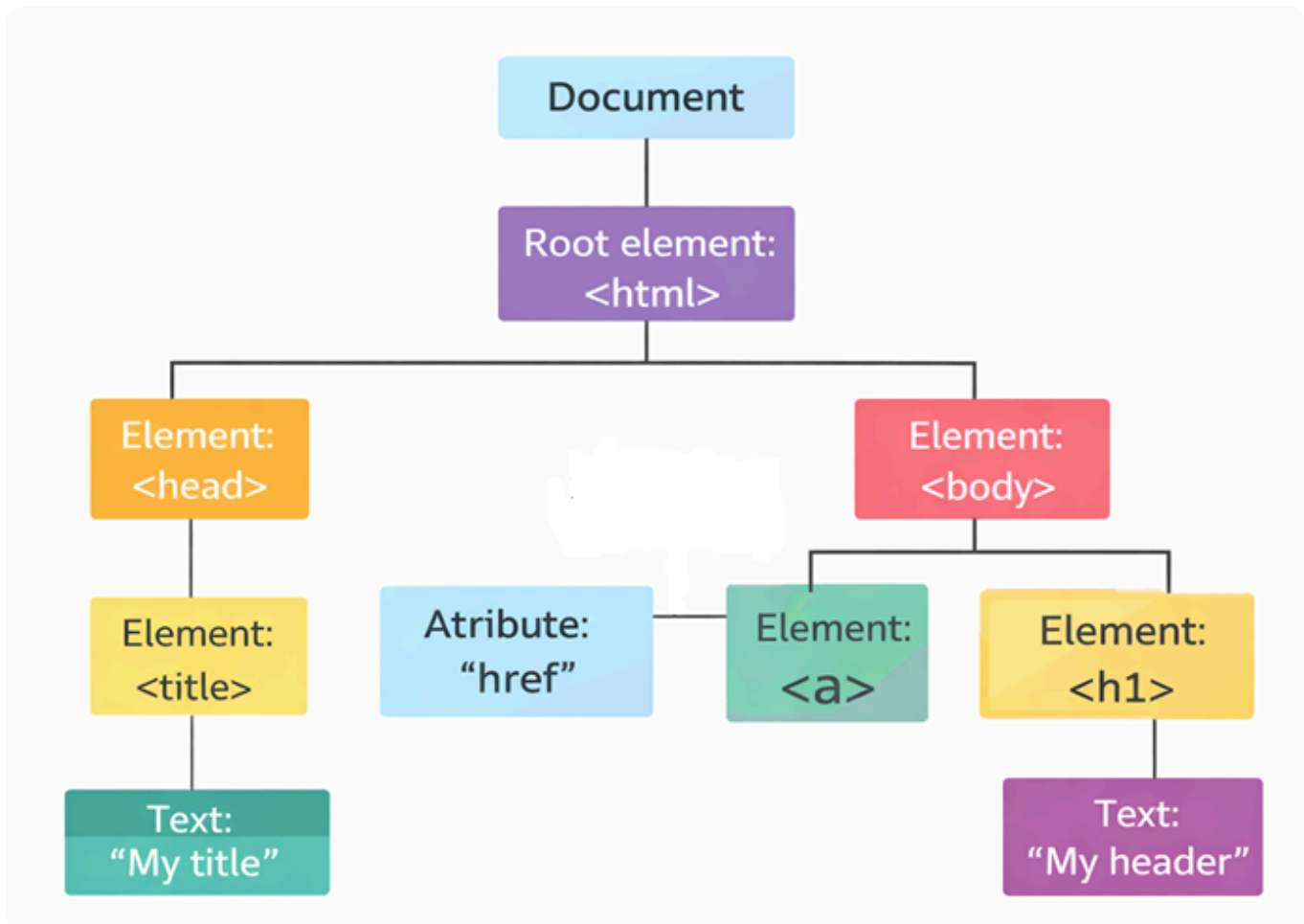
JAVASCRIPT

```
btn.onclick = "test";  
console.log(btn.onclick); // null
```

٣

Add and Remove Element from HTML

- في الاول شوفنا الصورة ده



- و اتكلمنا عن ان كل حاجة في الصورة ده اسمها node
- و ان js بتمثل ال document في شكل tree
- و كل element ليه children
- و اتكلمنا ان حتى ال text اللي بنكتبه جوا ال element ده عبارة عن node

- هنتعلم ازاى نعمل ال element ده ب js و ازاى نمسحه
- طب ايه الفائدة من كدة ؟
- ال html اللي بنكتبه في صفحة html هو في الاول و في الاخر عبارة عن حاجة ثابتة مبتتغيرش
- و بيمثل عموما الشكل الابتدائي للصفحة يعني ده اول حاجة اليوزر بيشفوها لما بيدخل الصفحة
- بس الطبيعي ان في حاجات بتتغير بناء على تفاعل اليوزر مع الصفحة ده
- عنصر يتغير عنصر يتضاف عنصر يتمسح

•

- ناخذ مثال عندنا صفحة بتعرض مثلا اسماء ال students اللي عندنا
- ايه اللي ممكن يكون ثابت في الصفحة ده و ايه اللي بيتغير
- العنوان بتاع الصفحة مش هيتغير
- ال List اللي هنكتب فيها اسماء ال students مش هتتغير
- لكن الاسماء نفسها اللي هي عناصر ال list ده هتتغير

نفذ المثال ده و نتعلم عليه

```
<!DOCTYPE html>
<html>
  <head>
    <title>HTML</title>
  </head>
  <body>
    <h1>Students</h1>
    <ul></ul>
    <script src="main.js"></script>
  </body>
</html>
```

- ده ال html المبدائي بتاعنا
- دلوقتي محتاجين نعمل العناصر بتاعت ال List
- ازاى نعمل عنصر html جديد ب js ؟
- بنستخدم فانكشن اسمها `createElement()` موجودة جوا ال `document` بنديها باراميتير اسم ال tag اللي عايزين نضيفه

```
const student1li = document.createElement('li');
const student2li = document.createElement('li');
const student3li = document.createElement('li');
```

- كدة احنا عملنا العناصر بس هي اتضافت كدة خلاص في html ؟
- لا احنا لسة مقولناش للكود يحطهم في html
- هم اتعملو و اتخزنو في variable بس

- طب عملنا عناصر طب هم ناقصهم حاجة
- هم دلوقتي عبارة عن عناصر بس مش العنصر ده بيبقى جواه text مثلا
- طب نقدر نضيف ال text ده ازاي ؟
- ال text ده عبارة عن node فاحنا هنعمل ال node الاول
- هنستخدم `createTextNode()` و بنديها باراميتر ال text اللي علوزين نخطه

JAVASCRIPT

```
const student1 = document.createTextNode("Ahmed");
const student2 = document.createTextNode("Mona");
const student3 = document.createTextNode("Sara");
```

- هل هم كدة اتضافو في html ؟ لا
- طب اتضافو جوا ال li ؟ لا
- هم لوحدهم كدة عملناهم و مضيفناهمش في حنة
- دلوقتي بقى هنضيفهم
- ال text عبارة عن child لل li فهنستخدم `appendChild()`
- ده بتبقى method جوا اي node بنعملها
- بنديها باراميتر node تانية

JAVASCRIPT

```
student1li.appendChild(student1);
student2li.appendChild(student2);
student3li.appendChild(student3);
```

- كدة ضيفنا ال text جوا ال li
- محتاجين نضيف بقى ال li جوا ال ul
- اول حاجة ال ul ده عبارة عن node برودو
- يعني هيكون جواها `appendChild()`
- طب عشان نستعمل `appendChild()` لازم يكون معنا ال ul نفسها

JAVASCRIPT

```
const studentsul = document.getElementsByTagName('ul')[0];
studentsul.appendChild(student1li);
studentsul.appendChild(student2li);
studentsul.appendChild(student3li);
```

- كدة كله اتضاف ل html

- طب لو عايزين نمسح li معين مثلا باللي جواه
- بنستخدم `removeChild()` بنحتاج وقتها يكون معنا ال parent بتاع ال element ده
- و بنديها ال element كباراميتر

```
studentsul.removeChild(student3li);
```

